

FEATURES

⚙️ Core Implementation & Features

1. Real-Time Data Entry

Hospital staff enter current values directly into the dashboard form, for example:

ICU total = 20	General total = 50	Emergency total = 10
ICU occupied = 16	General occupied = 20	Emergency occupied = 5

Clicking **⚙️ UPDATE DASHBOARD** triggers the full update pipeline (JS → JSON → POST /update → FastAPI → recalculation → refreshed dashboard).

2. Bed Availability & Occupancy Calculation

Available Beds = Total Beds - Occupied Beds
Occupancy % = (Occupied / Total) × 100

Example (ICU): 20 - 16 = 4 available, (16/20) × 100 = 80% occupancy.

3. Automatic Status Detection

Occupancy	Status
< 80%	🟢 NORMAL
≥ 80%	🟡 WARNING
= 100%	🔴 CRITICAL

The dashboard badge updates automatically — no manual interpretation needed.

4. Doctor & Nurse Resource Tracking

Each department (Emergency, ICU, General) tracks total vs. available staff:

Busy = Total - Available

Example — Doctors: Total = 11, Available = 8 → Busy = 3
Example — Nurses: Total = 20, Available = 13 → Busy = 7

5. Equipment Monitoring

Ventilators, ECG machines, and patient monitors follow the same in-use calculation:

In Use = Total - Available

Example: Ventilators: 10 total, 4 available → 6 in use

6. Rule-Based Recommendation Engine

The backend evaluates hospital data against predefined thresholds and generates alerts, for example:

Beds:	available = 0	→ CRITICAL
	occupancy ≥ 80%	→ WARNING
Doctors:	available = 0	→ CRITICAL
Nurses:	available ≤ 1	→ WARNING
Ventilators:	available ≤ 2	→ WARNING

This turns raw numbers into **actionable, human-readable recommendations** rather than just displaying data.

7. Live Dashboard Refresh

After any update, the frontend calls:

```
async function loadAll() {  
  await loadStatus();  
  await loadRecommendations();  
  await loadResourceChart();  
}
```

This refreshes status cards, progress bars, alert banners, and the Chart.js visualization in one flow, giving the feel of a live command center.

8. Visualization (Chart.js)

Resource utilization per department is rendered as a bar chart, e.g.:

ICU		80%
General		40%
Emergency		50%

9. Simulation Modes

- 🚑 **Emergency Surge** — pushes occupancy, staff availability, and equipment into shortage conditions to demonstrate how the system reacts under pressure.
- ✓ **Normal Condition** — restores predefined baseline values, useful for resetting between demos.

10. Login Page (Prototype-Level Access Control)

login.html → POST /login → credential check → redirect to index.html.
Demonstrates the *concept* of role/access control; credentials are currently hardcoded and this is explicitly **not** production-grade authentication.

11. API Testing & Docs

All endpoints are explorable and testable via the auto-generated Swagger UI at /docs, powered by FastAPI + Pydantic.

Is AI/ML Used?

Not currently. All optimization logic above is **rule-based** (if/else threshold checks), which keeps the system fast, transparent, and easy to explain. Planned extension: machine learning for demand forecasting, patient surge prediction, and predictive resource allocation.